Formalization of Gödel's Incompleteness Theorems in Basic Recursive Arithmetic (BRA)

Summary of an Agda development

May 2026

Abstract

We present a fully formalized proof of both Gödel's First and Second Incompleteness Theorems in a tree-based variant of Guard's Basic Recursive Arithmetic (BRA), executed in Agda 2.8 with `--safe --without-K --exact-split`. The development contains **zero postulates and zero holes**; `godelII : Deriv Con -> Deriv bot` typechecks end-to-end against the abstract Goedel II skeleton.

There is a single `Term` datatype (no separate `Tree` datatype), and "codes" are exactly the value-shaped Terms certified by an `IsValue : Term -> Set` predicate (built only from O and `ap2 Pair`). The map `reify : Tree -> Term`, written explicitly throughout the codebase for clarity at code/term boundaries, is the identity.

The headline results are:

- *thm11* : *Deriv G* $\to$ *Deriv* $\bot$ (Gödel I);
- *godelII* : *Deriv Con* $\to$ *Deriv* $\bot$ (Gödel II).

The development follows Guard's 1963 lecture notes [**?**]. The intermediate construction is a single first-order combinator *constr5* : Fun_1 realising Guard's "$F(x)$" (page 17); each step is given by an explicit equational derivation.

A key conceptual point this note emphasises (Section **??**) is that intermediate steps of Theorem 14's chain produce *equational* `Deriv`s of the form `atomic (eqn (thmT t) u)` where `u` is *not* `codeFormula P` for any P : *Formula*. Only the final `step5` conclusion has `u = codeFormula bot` (with $\bot$ genuinely closed). Guard's chain operates one level lower than Hilbert–Bernays–Löb: instead of internal provability claims about specific formulas, it manipulates substituted-codes through standard BRA inference rules. This makes the entire proof internal and equational — a feature of the framework, not a gap.

1 Introduction

The aim is a feasible, ASCII-only, syntax-first Agda formalization of Gödel's two incompleteness theorems following Guard's 1963 system of *Basic Recursive Arithmetic* [**?**].

1.1 Single-layer Term language with *IsValue*

In BRA2 there is one syntactic layer.

The `Term` language.

- *Term*: built from O (leaf), *var* n, $ap_1\ f\ t$, $ap_2\ g\ t_1\ t_2$.

- Function symbols: Fun_1 ($\{I, Fst, Snd, Comp, Comp_2, Z\}$, plus $KT\ \_$ as a defined function), Fun_2 ($\{Pair, Const, Lift, Post, Fan, IfLf, TreeEq, treeRec\}$).
- *Formula*: *atomic* (*eqn* $\ell\ r$), *not* $\_$, $\_$ *imp* $\_$.
- *Deriv* F: inductive family of derivations. Constructors for the algebraic axioms (ax_*), the equality axioms (Guard's Ax 4–7), three propositional axioms (axK, axS, $axNeg$ in Lukasiewicz form), modus ponens (mp), substitution ($ruleInst$), and binary tree induction ($ruleIndBT$).

Codes are value-shaped Terms. "Codes" are Terms whose value-shape is certified by

$$IsValue : Term \to Set,$$

with constructors $valO : IsValue\ O$ and $valNd\ a\ b\ v_a\ v_b : IsValue\ (ap_2\ Pair\ a\ b)$ (when $IsValue\ a$ and $IsValue\ b$).

The encoding maps now have type

$$code : Term \to Term, \qquad codeFormula : Formula \to Term,$$

together with structural-soundness lemmas $code\_isValue : (t : Term) \to IsValue\ (code\ t)$, $codeFormula\_isValue$, $codeF_1\_isValue$, $codeF_2\_isValue$, and $natCode\_isValue$ (BRA2/Term.agda, BRA2/Formula.agda). Every primitive constructor of an encoding produces a value-shaped Term.

Notational aliases `Tree`, `lf`, `nd`, `reify`. For continuity with Guard's prose, BRA2/Term.agda exposes

$$Tree := Term, \quad lf := O, \quad nd\ a\ b := ap_2\ Pair\ a\ b, \quad reify\ t := t.$$

The Layer-1/Layer-2 distinction is recorded as the $IsValue$ predicate; syntactically the layers are unified.

Reflection layer (object-level operations on codes). Functors that reason about BRA syntax internally:

- $cor : Fun_1$: defined as $Rec\ stepCor$ (BRA2/Cor.agda). Spec: $corOfReify : (t : Term) \to IsValue\ t \to Deriv\ (ap_1\ cor\ t = code\ t)$. Operationally, cor takes a value-shaped Term (= a "numeral") and returns its code.
- $sub : Fun_2$, $subT : Fun_2$: coded substitution (BRA2/Sub.agda, BRA2/SubT.agda), now indexed by $IsValue$ on the substituent.
- $thmT : Fun_1$: the proof-checker (Section **??**).

Reflection theorem.

$$encode : (P : Formula) \to Deriv\ P \to \Sigma\ Term\ \big(\lambda y.\ \Sigma\ (IsValue\ y)\ \big(\lambda\_.\ Deriv\ (thmT\ y = codeFormula\ P)\big)\big).$$

The $IsValue\ y$ field certifies that every encoded proof tree is value-shaped, so $cor\ y$ provably reduces to $code\ y$ at the witness returned by $encode$. This certification is exactly what discharges the parametric `cor x` in the chain at the closure (see Section **??**).

1.2 Notational convention: code-quotes and hats

Code-quotes $\ulcorner\cdot\urcorner$. We use $\ulcorner E\urcorner$ throughout for "the code (value-shaped Term) of E":

$$\ulcorner t\urcorner := \mathit{code}\ t \quad (t : \mathit{Term}), \qquad \ulcorner F\urcorner := \mathit{codeFormula}\ F \quad (F : \mathit{Formula}).$$

Whenever a Guard-style expression mixes object-level and encoded material (e.g. $f(\underline{x})$), we keep the code-quote explicit: the formal object is the Term $\ulcorner f(\underline{x}) = \underline{f(x)}\urcorner$, never the unquoted equation. See Section **??** for the full caveat.

Hats $\hat{}$ and $\widehat{\ }$. We write $\hat{x}$ as a shorthand for $\mathit{cor}\ x$, the numeral-encoding combinator applied to a Term x. This lets us write e.g. $\mathit{th}\ \hat{x}$ for $\mathit{ap}_1\ \mathit{thmT}\ (\mathit{ap}_1\ \mathit{cor}\ x)$ and $\mathit{sub}\ \hat{x}\ \hat{y}$ for $\mathit{ap}_2\ \mathit{sub}\ (\mathit{ap}_1\ \mathit{cor}\ x)\ (\mathit{ap}_1\ \mathit{cor}\ y)$.

For a Guard-style expression E, $\widehat{E}$ denotes the *substituted-code* of E: the Term obtained by taking the code of E's pattern (with variable slots in place of $\hat{}$-marked positions) and sb-substituting the $\hat{}$-encoded numerals into those slots. E.g. $\widehat{\mathit{th}\ \hat{x}} = \mathit{ap}_2\ \mathit{Pair}\ \mathit{tagAp}_1\ (\mathit{ap}_2\ \mathit{Pair}\ (\mathit{codeF}_1\ \mathit{thmT})\ (\mathit{ap}_1\ \mathit{cor}\ x))$. At a free Term variable x, this substituted-code is in general not $\ulcorner P\urcorner$ for any $P : \mathit{Formula}$ (Section **??** discusses this in detail); for value-shaped x it BRA-reduces to a genuine codeFormula via *corOfReify*.

2 The provability predicate *thmT*

The Gödel-style "*th*" provability predicate is realized in BRA by a single Fun_1-functor:

$$\mathit{thmT}\ =\ \mathit{Rec}\ \mathit{stepProtoWrapped}\ :\ \mathit{Fun}_1.$$

Here *stepProtoWrapped* is a Fun_2 assembled from a 40-deep linear *IfLf* cascade (BRA2/Thm/StepProto.agda and BRA2/Thm/ThmT.agda), one branch per primitive proof rule: ten equational axioms of Group I, eight Group II axioms, nine equality and propositional rules, and three rules of inference (*mp*, *ruleInst*, *ruleIndBT*).

The *key* property of *thmT* is the encoding-soundness lemma *encode* above, proved by induction on the *Deriv* derivation in BRA2/Thm/ThmT.agda's *encodeRich*.

2.1 Self-substitution closure

thmT is a closed combinator:

$$\mathit{thClosed} : \forall x\ r, \quad \mathit{substF}_1\ x\ r\ \mathit{thmT} = \mathit{thmT}$$

and its encoding is var-free:

$$\mathit{codeF1Th\_noVar} : \forall x\ \mathit{repl}, \quad \mathit{codedSubst}\ \mathit{repl}\ (\mathit{code}\ (\mathit{var}\ x))\ (\mathit{codeF}_1\ \mathit{thmT}) = \mathit{codeF}_1\ \mathit{thmT}.$$

3 Gödel's First Incompleteness Theorem (Theorem 11)

3.1 The diagonal sentence

Following Guard, define

$$\begin{aligned} F_{\mathrm{pre}} &:= \neg\big(\mathit{thmT}\ v_1\ =\ \mathit{sub}\ v_0\ v_0\big), \\ j &:= \mathit{codeFormula}\ F_{\mathrm{pre}} \quad \text{(a closed value-shaped Term)}, \\ G &:= F_{\mathrm{pre}}[v_0 := j]. \end{aligned}$$

The sentence G has v_1 free; informally, G asserts of itself that no proof tree v_1 proves the formula whose code is $j[v_0 := j]$.

The certificate $j\_isValue : IsValue\ j$ is exposed inside the `Diagonal` abstract block (BRA2/Thm11Diagonal.agda It propagates through $Outer.GodelII$ and is what makes `corOfReify j j_isValue` usable downstream.

(For continuity with prior notation, the Agda codebase additionally defines `i := reify j`; since `reify` is the identity, $i = j$. We use j throughout this paper.)

3.2 Theorem 11

Theorem 1 (`thm11`, Gödel I)**.** *thm11* : *Deriv* $G \to$ *Deriv* $\bot$, *where* $\bot :=$ *atomic* (*eqn trueT falseT*), *trueT* $:= O$, *falseT* $:= ap_2\ Pair\ O\ O$.

Sketch. From any π : *Deriv* G extract its encoding (y_d, v_d, E_1) where v_d : *IsValue* y_d and E_1 : *Deriv* (*thmT* y_d = *codeFormula* G). A separate diagonal lemma (BRA2/Thm11Diagonal.agda) yields N : *Deriv* $\neg$(*thmT* y_d = *codeFormula* G), and $ax_{ExFalso}$ + two *mp*'s collapse E_1 and N to *Deriv* $\bot$. □

4 Gödel's Second Incompleteness Theorem (Theorem 14)

Theorem 2 (`godelII`, Gödel II)**.** *godelII* : *Deriv Con* $\to$ *Deriv* $\bot$, *where Con* $:= \neg$(*thmT* v_0 = *codeFormula* $\bot$).

4.1 Caveat: "$f(\underline{x}) = \underline{f(x)}$" is not a BRA formula

A structurally crucial point, easy to miss in Guard's notation: the expression

$$f(\underline{x}) \;=\; \underline{f(x)}$$

is **not** a BRA formula. Guard's underline $\underline{(\cdot)}$ takes a value and returns its numeral term — well-defined only at the meta-level for closed numerical inputs. For a variable x (the case Theorems 12–14 actually need), neither $\underline{x}$ nor $\underline{f(x)}$ has any BRA referent: there is no operator in BRA's term language that, given an arbitrary Term, returns "the numeral of its value."

The only formal object that *does* correspond to Guard's expression is its *code*, the Term

$$\ulcorner f(\underline{x}) \;=\; \underline{f(x)} \urcorner,$$

which Definition 12 of Guard's notes *defines* by stipulation. In BRA2 this Term is built parametrically as $codeFTeq_1\ f\ x$, with *cor* x filling the slot for the code of x's numeral and *cor* $(f\,x)$ filling the slot for the code of $f(x)$'s numeral.

When x is a free Term variable, neither *cor* x nor *cor* $(f\,x)$ reduces (the defining equation *corOfReify* fires only on *IsValue* inputs), so $codeFTeq_1\ f\ x$ contains ap_1-headed subterms literally. Codes (in the sense of *codeFormula* P) are *IsValue*-shape, so $codeFTeq_1\ f\ x$ at variable x is **not** of the form *codeFormula* P for any P : *Formula*.

4.2 Theorem 12 — the encoded "*th* f" clause

For every Fun_1 functor f, Theorem 12 produces a $Fun_1\ D_f$ such that *thmT* evaluated at $D_f\ x$ is provably equal to the encoded form of "$f(\underline{x}) = \underline{f(x)}$":

Theorem 3 (*thm12_F*$_1$)**.** *thm12_F*$_1 : (f : Fun_1) \to Thm12\_F_1\_Result\ f$ *where* $Thm12\_F_1\_Result\ f$ *packages* $D_f : Fun_1$ *with proof* $: (x : Term) \to Deriv\big(thmT\ (D_f\ x)\ =\ codeFTeq_1\ f\ x\big)$.

The conclusion's RHS is the literal Pair-shape encoded equation. An analogous statement for Fun_2. Structural recursion on f with one clause per constructor (15 total), BRA2/Thm12.agda module *FromBridges*.

4.3 Theorem 13 — under hypothesis $f\ x = y$

Lemma 4 (*thm13_F*$_1$)**.** *For every* $f : Fun_1$ *and every Theorem 12 result* r_{12} *for* f, $thm13\_F_1\ f\ r_{12}\ x\ y :$ $Deriv\ (f\ x = y) \to Deriv\ (thmT\ (D_f\ x) = codeFTeq_{1,\mathrm{Hyp}}\ f\ x\ y)$.

The conclusion replaces $cor\ (f\,x)$ by $cor\ y$. One-line proof: *ruleTrans* on $r_{12}.proof\ x$ with $cong_R\ Pair$ on $cong_1\ cor$ of the hypothesis.

4.4 Theorem 14 — the consistency-to-bot construction

The contract abstracted in BRA2/Thm14Abstract.agda asks for:

1. a Fun_1-functor *constr5* such that $thmT\ (constr5\ x)$ syntactically encodes $\bot$;
2. a parametric internal-implication

$$step5 : (x : Term) \to Deriv\big((thmT\ x = codeFormula\ G)\ imp\ (thmT\ (constr5\ x) = codeFormula\ \bot)\big).$$

The closure *closureToG* instantiates *step5* at v_1, *ruleInst*s *Con*, modus-ponenses, transports through *subIIeqcG* and G_{unfold}, arriving at *Deriv G*. Composing with *thm11* gives *godelII*.

4.4.1 Construction of *constr5*

Two pre-existing tautologies, encoded:

- $t := (v_0 = v_2)\ imp\ ((v_1 = v_2)\ imp\ (v_0 = v_1))$ (transitivity);
- $t' := (v_0 = v_1)\ imp\ ((\neg(v_0 = v_1))\ imp\ \bot)$ (ex falso).

Both BRA-derivable (BRA2/Thm14Numerals.agda); *encode* yields value-shaped Terms $tterm, t'term$ certified by $t\_term\_isValue$ / $t'\_term\_isValue$ (these certificates are load-bearing for the chain — see Section **??**).

The chain, paralleling Guard's page 17:

$$\begin{aligned}
f_part(x) &:= ruleInst\ v_0\ (th\ \hat{x})\ \big(ruleInst\ v_1\ (sub\ i\ i)\ (ruleInst\ v_2\ cor\ G\ tterm)\big),\\
g_part(x) &:= mp\big(mp(f_part(x), D_{thmT}\ x), D_{sub}\ i\ i\big),\\
K_part(x) &:= ruleInst\ v_1\ (cor\ x)\ x,\\
h_part_skel(x) &:= ruleInst\ v_0\ (th\ \hat{x})\ \big(ruleInst\ v_1\ (sub\ i\ i)\ t'term\big),\\
constr5_{\mathrm{final}}(x) &:= mp\big(mp(h_part_skel(x), g_part(x)), K_part(x)\big).
\end{aligned}$$

4.4.2 The five steps

(BRA2/Th14*.agda.)

Step 1 — *step1_l* Theorem 13 applied to $thmT$.

Step 2 — *step2_l* Theorem 13 applied to *sub* at $(j, j, cor\ G)$ under closed hypothesis *subIIeqcG*.

Step 3 — *step3_l* Three sb-applications on t, canonicalised, then two *mp* dispatches.

Step 4 — *K_part_l* One sb-application at v_1 on G_{norm} via the $PrAtX\ x$ hypothesis to swap $thmT\ x \mapsto cor\ G$.

Step 5 — *step5_l* Two outer *mp* dispatches at $constr5_{\text{inner,final}}$ and $constr5_{\text{final}}$.

5 What is going on at the encoded layer: a remarkable internal proof

5.1 The phenomenon

The chain $step1\_l, \ldots, step5\_l$ proves *Deriv*s of the schematic form

$$Deriv\ \big(PrAtX\ x\ imp\ atomic\ (eqn\ (thmT\ t_i)\ u_i)\big),$$

where t_i, u_i are Terms depending on x. Several of these intermediate u_i are **not** of the form *codeFormula* P for any $P : Formula$.

A representative example: *step3_l* (BRA2/Th14Step3.agda) concludes

$$Deriv\ \big(PrAtX\ x\ imp\ atomic\ (eqn\ (thmT\ (g\_part\ x))\ ap_2\ Pair\ \widehat{(th\ \hat{x})}\ \widehat{sub\ i\ i})\big),$$

with

$$\widehat{th\ \hat{x}}\ =\ ap_2\ Pair\ tagAp_1\ (ap_2\ Pair\ (codeF_1\ thmT)\ (ap_1\ cor\ x)).$$

At a variable x (e.g. $x = v_1$), $ap_1\ cor\ x$ does not reduce: *corOfReify* requires *IsValue* x, and v_1 is not value-shape. The Term $ap_1\ cor\ x$ has an ap_1 head; codes are exclusively *IsValue*-shape (built from O and $ap_2\ Pair$). Therefore $u_3 = ap_2\ Pair\ \widehat{(th\ \hat{x})}\ \widehat{sub\ i\ i}$ is not in the image of *codeFormula*.

The same phenomenon recurs in $step4, step5_{\text{intermediate}}, h\_part\_skel\_l$. In each case, u_i is a *substituted-code* — the result of inserting $cor\ x$ into a variable slot of the code of an open formula.

Yet $Deriv\ (atomic\ (eqn\ t\ u))$ is perfectly well-formed for arbitrary Terms t, u — the equational predicate *eqn* does not require u to be a code of a formula. The chain composes via the standard inference rules (*mp*, *ruleTrans*, $cong_*$, *axS*, *axK*) without ever needing the intermediate u_i to be *codeFormula* P.

5.2 Why this works: substituted-codes vs. codes-of-formulas

There are two distinct readings of Guard's notation $\ulcorner f(\underline{x}) = \underline{f(x)} \urcorner$:

(A) Code-of-substituted-formula. Read $\underline{x}$ as $cor\ x$ (a BRA Term), and form the BRA formula $atomic\ (eqn\ (ap_1\ f\ (cor\ x))\ \ldots)$. Take its *codeFormula*. The inner slot becomes $code\ (cor\ x)$ (= the code of the BRA term $cor\ x$).

(B) Substituted-code. Take the open formula $atomic\ (eqn\ (ap_1\ f\ v_n)\ \ldots)$, code it, then *sb*-substitute $cor\ x$ into the $\ulcorner v_n \urcorner$ slot. The inner slot becomes $cor\ x$ itself.

At a free variable x these readings disagree: (A) places *code* (*cor* x) in the slot, (B) places *cor* x. Theorem 12's $codeFTeq_1$ uses reading (B) — the substituted-code form. This is forced by the structure of D_f, which is built from $Comp_2\ Pair\ \ldots\ cor$, producing *cor* x in its evaluation directly.

The two readings coincide **when x is value-shaped**: *corOfReify* then gives *cor* x = *code* x, and *Lemma codeCommutesFormula* (BRA2/CodeCommutes.agda) gives

$$code\ (substF\ n\ t\ P)\ =\ codedSubst\ (code\ t)\ (code\ (var\ n))\ (codeFormula\ P).$$

So at value-shape x, the substituted-code (B) is BRA-equal to *codeFormula* ($substF\ n\ x\ P_{\text{open}}$) — a genuine code-of-formula. At variable x this collapse fails, and (B) remains a hybrid Term.

5.3 The chain's discharge mechanism

Why does Goedel II close even though step 3 etc. produce hybrid RHS?

1. *Internal composition stays at the equational level.* The chain's mp/cong/ruleTrans steps operate on *eqn* $t\ u$ for arbitrary t, u; they do not require u to be *codeFormula* P. All intermediate Derivs are honest BRA derivations producing well-typed atomic equations.

2. *step5 's conclusion is genuinely codeFormula* $\bot$. At step 5 the final RHS is *codeFormula* $\bot$, where $\bot$ is closed. No *cor* x remains in the conclusion — the Comp2-of-KT-of-codeFormula-bot construction in $constr5_{\text{final}}$ is a constant in x modulo Lift, so its $thmT$-image is a closed code by construction.

3. *Closure transports through notEqTransport at a value-shape witness. closureToG* instantiates *step5* via *ruleInst* at the encode-witness of G's proof. By the strengthened *encode* contract (returning *IsValue*), this witness is a value-shaped Term. At such a witness, *cor* x inside any intermediate substituted-code rewrites to *code* x via *corOfReify*, and the substituted-codes collapse to genuine *codeFormula*s of substituted formulas.

The interleaving — carry hybrid forms through internal-implication chains, collapse at the closure — is what lets the proof be parametric over (x : *Term*) rather than meta-iterated over closed numerals.

5.4 The projective specification of *mp*

A natural question raised by the chain is: what *exactly* is the specification of *mp* as embedded in $thmT$, given that step 3 applies *mp*-internalisation to inputs that are not *codeFormula* (P *imp* Q) for any P, Q : *Formula*?

A naive declarative reading would assert

$$mp\big(codeFormula\ (P\ imp\ Q), codeFormula\ P\big)\ =\ codeFormula\ Q.$$

This is the soundness reading at well-typed inputs. But it is *not* what BRA's $thmT$ actually computes, and it is not what step 3 uses. The actual specification is the verifying-body cascade in BRA2/SoundMpVProof.agda:

> Given a coded proof tree $arg = ap_2\ Pair\ tagMp\ (ap_2\ Pair\ y_1\ y_2)$, $thmT(arg)$ dispatches via $body\_mp\_v(a, b) = Post\ verifierF_1\ Pair\ a\ b$ where $verifierF_1$ first computes $thmT\ y_1$ and $thmT\ y_2$, then performs *two TreeEq* checks:

1. *outerCheck*: $Fst(thmT\, y_1) \stackrel{?}{=} tagImp$,
2. *innerCheck*: $thmT\, y_2 \stackrel{?}{=} Fst(Snd(thmT\, y_1))$.

If both succeed, the verifier returns $Snd(Snd(thmT\, y_1))$, the consequent slot. If either fails, it returns $codeTriv = falseT$, the safe default.

Soundness. The two *TreeEq* checks together ensure that the consequent extracted is the consequent of *this specific implication* applied to *this specific antecedent*. Without the inner check $thmT\, y_2 = Fst(Snd(thmT\, y_1))$, BRA's *mp*-dispatch would project the second slot of an arbitrary *tagImp*-headed Term, with no relation to y_2, and $thmT$ would fail to be a sound proof-checker (the encoding-soundness lemma in BRA2/Thm/ThmT.agda's *encodeRich* would not go through). The inner check is what makes the *mp* clause of *encodeRich* valid: it certifies, equationally inside BRA, that the two sub-proofs y_1, y_2 are compatible at the expected antecedent.

Why this still works on substituted-codes. The two *TreeEq* checks are *equational comparisons on Terms*, not type-checks on Formulas. Whatever sits in the inner slots of $thmT\, y_1$ and $thmT\, y_2$ — $codeFormula\, P$ and $codeFormula\, Q$, or $cor\, x$ interleaved with tag constants, or any value-shaped Term — only matters in that the inner check fires correctly: the antecedent slot of $thmT\, y_1$ must be *TreeEq*-equal to $thmT\, y_2$.

In step 3 of Theorem 14, the equational chain is set up precisely so that this match holds: each *mp*-internalised step ensures, via the previous step's *Deriv*, that the antecedent slot of the current implication's substituted-code is exactly the consequent of the previous step. So the two *TreeEq* checks both fire *equationally*, with both sides being substituted-codes, and the dispatch returns the consequent's substituted-code. The chain composes precisely because both readings of *mp* — the "well-typed Formula" and "well-shaped substituted-code" cases — are handled by the same projective primitive-recursive computation.

The result is again a substituted-code, fed into the next step the same way:

$$mp\text{-}body\big(\text{substituted-code-of-implication}, \text{substituted-code-of-antecedent}\big) = \text{substituted-code-of-consequent},$$

equationally, with no claim about Formula content but with the soundness-relevant antecedent-match certified by *TreeEq*.

The two readings of *mp*'s specification stand in a clear hierarchy:

Projective (what BRA actually computes). The *Fst*/ *Snd* projection above, parametric in the inner Terms. Total on all Term inputs of the right outer shape, producing a Term output by syntactic projection.

Declarative (what soundness justifies). At literal $codeFormula\, (P\ imp\ Q)$ inputs, the projective behaviour delivers $codeFormula\, Q$. This is what makes $thmT$ a sound proof-checker (*encodeRich*'s *mp* clause).

The soundness reading factors through the projective reading: BRA's *mp* primitive only knows about Term shape, and Formula-content correctness is the *consequence* for the well-shaped subset of inputs. Theorem 14 step 3 uses the projective reading directly; the final closure converts to the declarative reading by instantiating at a value-shape witness.

The same observation lifts to the other "soundified" rule dispatchers in the chain (*ruleSym*, $cong_1$, $cong_L$, $cong_R$, *ruleTrans*, *ruleInst*, $ruleInst_2$, *ruleIndBT*). Each is specified as a primitive-recursive Term projection on a shape-discriminated input; each delivers a Formula-content interpretation only on its well-shaped subset. The verifying-body construction ($body\_X\_v = Post\ verifierX\ Pair$,

Section "Sound *thmT*" of `README.md`) is precisely what enforces the safe default *codeTriv* on malformed inputs without spoiling the projective behaviour on well-shaped ones.

In one line: *BRA's mp is a Term projection that happens to internalise Formula modus ponens at well-typed inputs.* The chain in step 3 uses the projection-on-Term reading; the internal-modus-ponens reading is what justifies it semantically at the value-shape closure. Both readings are valid specifications; the projective one is strictly more general and is what makes the chain compose at variable x.

5.5 Comparison with Hilbert–Bernays–Löb

Standard textbook Gödel II proofs go through provability statements:

$$\text{D1} :\vdash A \implies \vdash \Pr(\ulcorner A \urcorner), \quad \text{D2} :\vdash \Pr(\ulcorner A \to B \urcorner) \to \Pr(\ulcorner A \urcorner) \to \Pr(\ulcorner B \urcorner), \quad \text{D3} :\vdash \Pr(\ulcorner A \urcorner) \to \Pr(\ulcorner \Pr(\ulcorner A \urcorner) \urcorner).$$

Every premise is a provability statement *about* a code of a specific formula. The chain stays at the level "F is provable" for various F. Reflection (D3) is needed to handle the iteration.

Guard's BRA chain operates one level lower: at the level of *equations* $thmT\ t = u$ for arbitrary Terms t, u. Reflection (D3-analogue) is realised internally and parametrically, without reference to any specific formula:

- Theorem 12 is the parametric "D_f is the encoded image of f" — one statement, all f.
- Theorem 13 lifts external equations to encoded form, parametric in the equational hypothesis.
- Theorem 14 chains substituted-codes through *mp/ sb/ruleInst*, never invoking a separate "$\Pr(\ulcorner \Pr(\ulcorner G \urcorner) \urcorner)$" reflection axiom. *step5* is an equational consequence of Theorem 12's defining equations under hypothesis, not a provability claim about a specific formula.

5.6 Remarkable consequences

1. **Avoids meta-reflection.** No $\Pr(\ulcorner \Pr(\ulcorner F \urcorner) \urcorner)$ axiom is invoked. The analogous internalisation falls out of *thmT*'s defining equations + Comp2/KT/cor compositions.
2. **Fully equational.** Goedel II reduces to BRA equational reasoning + *mp/axS/axK/ axNeg/axContrapos*. No specialised "provability logic" is needed.
3. **Parametric over** (x : *Term*)**.** Each step lemma works for any Term x (variable, application of cor to a variable, anything). Standard HBL proofs are usually meta-iterated over each closed witness.
4. **The intermediate Derivs are genuine BRA proofs that do not correspond to "BRA proves formula F" for any single F.** This is the structurally interesting feature: the chain proves *equational* statements between Terms, with the "provability" interpretation recovered only at the closure.

The cost is bookkeeping that the development makes explicit: the substituted-code RHS at variable x requires *IsValue* certificates (on j, on encode-witnesses, on t_term/t'_term) propagated through to the closure. These certificates are the formal counterpart of Guard's silent assumption that "$\underline{x}$" for variable x is sensible because x will eventually be instantiated at a closed numeral.

In short: *Goedel II in BRA is an equational proof at the code layer, internalised via primitive recursion, with the provability-logic story emerging only at closure.* Guard's 1963 formulation has this character intrinsically; HBL-style proofs do not. The Agda dev makes the distinction visible.

6 Methodology

6.1 Constraints maintained throughout

- `--safe --without-K --exact-split` on every file.
- ASCII only. No Unicode.
- No postulates. No holes. No `with`-abstraction. No dot patterns (one exception: `.O` / `.a .b` in *IsValue* pattern matches, which are the canonical syntax for matching against an indexed family's index).
- No new *Deriv* axioms beyond Guard's primitive list.

6.2 Verification

```
$ agda --safe BRA2/GoedelIIFull.agda
$ echo $?
0
$ ls BRA2/GoedelIIFull.agdai
BRA2/GoedelIIFull.agdai
$ grep -rn "^postulate" BRA2/
$ # (empty)
```

7 Discussion

The two Gödel theorems in BRA2:

- **Gödel I.** If the diagonal sentence G is provable, then $\bot$ is provable.
- **Gödel II.** If the consistency formula *Con* is provable, then $\bot$ is provable.

The intermediate functions $constr5_{\text{final}}$ and the parametric step lemmas *step1_l*–*step5_l*, *K_part_l*, *h_part_skel_l* are *first-order* in the sense of recursive-arithmetic definability: each is a closed combinator built from primitive Fun_1 / Fun_2 symbols, and each *Deriv* derivation is given by an explicit equational proof in the BRA system. No external soundness theorem is invoked.

7.1 Complexity calibration: ramified-elementary fragment

Although BRA's term language admits unrestricted primitive recursion on binary trees (the *treeRec* constructor of Fun_2), the entire chain reachable from `godelII : Deriv Con -> Deriv bot` in BRA2/GoedelIIFull.agda lies within the *ramified-elementary* fragment in the sense of Leivant [**?**]. Specifically, every functor used in the closure is well-typed under a stratification $\Omega^k\theta \to \theta$ with $k \leq 2$, where θ is the base type of binary-tree-shaped Terms and $\Omega\theta$ is the type of data supporting recurrence over θ in Leivant's discipline (paper III, Section 2.3, equation (4)).

Inventory of recurrence-using functors. A grep over the closure confirms that across `Cor.agda`, `SubT.agda`, `Sb.agda`, `Thm/ThmT.agda`, `Thm14Constr5Final.agda`, `Th14Step5.agda`, and `Th12TreeRecInternal.a` the constructor *treeRec* appears in exactly one syntactic definition (`Th12TreeRecInternal.agda:396`), plus the derived combinators $Rec\ s = Comp_2\ (treeRec\ Z\ s)\ Z\ I$ and $RecP\ s = treeRec\ Z\ s$. The headline functors unfold as follows.

- $cor = Rec\ stepCor$: one *treeRec*, level $\Omega\theta \to \theta$. *stepCor* is a *Fan*/*Lift*/*KT*/*Post*/*Pair* composite, no recurrence.
- $subT = RecP\ stepSubT$: one *treeRec*, level $\theta \times \Omega\theta \to \theta$. *stepSubT* uses only *Fan*, *IfLf*, *Lift*, *Post*, *Comp*.
- $sb = Fan\ \ldots\ subT$: a single *Fan* wrapping *subT*, level $\theta \times \Omega\theta \to \theta$ via *subT*.
- $thmT = Rec\ stepProtoWrapped$: one *treeRec*, level $\Omega\theta \to \theta$. *stepProtoWrapped* is the 40-tag *IfLf*/*Fan*/*Lift*/*KT*/*TreeEq* cascade, no recurrence.
- $D_{thmT} = thm12\_F1\ thmT.Df$: depth-2 use of *treeRec*. The outer *treeRec lf_emitter step_emitter* walks the input encoding; inside *step_emitter*, the original *treeRec f s* is invoked as $Comp_2\ (treeRec\ f\ s)\ Fst_1$ i.e. on a *Fst*/*Snd*-extracted *critical sub-argument* of the outer recurrence argument (BRA2/Th12TreeRecInter 226, 354–356). This is Leivant's allowed pattern: the critical args $\vec{a}$ in $g_{c_i}(\vec{x})(\vec{a})(\ldots)$ live at type $\Omega\theta$ when the outer recurrence argument is at $\Omega\theta$, so re-invoking $f : \ldots, \Omega\theta \to \theta$ on them is well-typed. Level: $\theta \times \Omega^2\theta \to \theta$ when seen one level higher; safely $\Omega\theta \to \theta$ when applied with the outer recurrence argument at $\Omega^2\theta$.
- $D_{sub\_at\_ii}$, g_{part}, h^{skel}_{part}, K_{part}, $constr^{final}_5$: $Comp_2$-of-*Pair* / *KT* compositions of the above. No new *treeRec*.

Maximum nesting depth: 2. Across the entire closure, no *treeRec* ever takes its recurrence argument from a recursion *result* of another *treeRec*. The single nested case (D_{thmT}) is the structural-recursion-on-critical-arg form, which is well-typed at level $\Omega^2\theta \to \theta$ in Leivant's discipline. Every functor in the closure terminates in time elementary in the size of its input.

Implication: Gödel II in elementary BRA. By Leivant's main theorem (Paper III, Theorem at p. 210), the functions definable by ramified higher-type recurrence at types of order at most k are exactly the elementary (Kalmar) functions $\mathcal{E}_3$. The BRA2 closure thus computes only elementary functions on Tree-codes. This is sharp: by Wilkie–Paris and Buss's formalisation [**?**], elementary arithmetic $EFA = I\Delta_0 + \mathrm{EXP}$ already proves the second incompleteness theorem; nothing weaker than EXP suffices, because Hilbert–Bernays–Löb condition D3 (Σ_1-completeness on closed numerical statements) requires at least exponential encoding-arithmetic. BRA2 sits at exactly this threshold: enough to encode and substitute proofs, no stronger. The headline result of the development can therefore be sharpened from "Gödel II in BRA" to:

> **Theorem (calibrated headline).** Gödel's second incompleteness theorem is provable in the ramified-elementary fragment of BRA — the $\Omega^k\theta \to \theta$ fragment with $k \leq 2$ in Leivant's higher-type ramification — with no use of unrestricted higher-type primitive recursion.

This calibration is a post-hoc certificate: it does not require any change to the existing `--safe` Agda development, only the observation that none of the constructed functors exit the elementary fragment. The audit (file inventory, *treeRec* occurrence count, depth-2 nesting check) is a finite verification on the seven files listed above.

7.2 Induction calibration: atomic-predicate tree induction only

The function-class audit of Section **??** measures *which functors* are needed. A complementary audit measures *which induction rules and at what predicate complexity* are needed. This second audit is even sharper.

BRA's primitive induction rule *ruleIndBT* (and its diagonal variant $ruleIndBT_2$, used for *TreeEq*'s pair-of-pairs recursion; see BRA2/Deriv.agda lines 218–253) accepts an *arbitrary Formula* as the induction predicate — atomic equations, negations, implications, and any nesting thereof.

Inventory of induction call sites. A clean recheck of `BRA2/GoedelIIFull.agda` under `--safe` (147 files in the closure) lists exactly three *ruleIndBT* / $ruleIndBT_2$ call sites, with the following induction predicates.

1. `TreeEqReflParam.agda:158`. $motiveTreeEqRefl = atomic\ (eqn\ (ap_2\ TreeEq\ (var\ 0)\ (var\ 0))\ O)$. *Atomic equation.*
2. `Th12TreeEqUniv.agda:72` ($ruleIndBT_2$). $P_{Th12\_TreeEq} = atomic\ (eqn\ (ap_1\ thmT\ (ap_2\ D_{TreeEq}\ (var\ 0)\ (var\ 1$ *Atomic equation.*
3. `Th12TreeRecInternal.agda:1786`. $P_{Th12\_treeRec} = atomic\ (eqn\ (ap_1\ thmT\ (ap_2\ D_2^{treeRec_{f,s}}\ (var\ 1)\ (var\ 0)))$ *Atomic equation.*

Every induction predicate in the closure is an atomic equation. No induction over an implication, no induction over a negation, no induction over a compound boolean combination, no nested predicate of any kind. Restricting BRA's *ruleIndBT* to the atomic-predicate fragment — *IndBT-atomic* — the entire BRA2 proof of Goedel II survives unchanged.

Significance. This is the BRA-language analog of *open induction* (Shepherdson's IOpen) / *quantifier-free induction*, specialised to BRA's vocabulary of primitive recursive functors. Since BRA has no quantifiers in its formula language at all, and the induction predicates here are not even disjunctions or implications but single equations between primitive-recursive Term-expressions, the relevant fragment sits *strictly below* EFA $= I\Delta_0 + \mathrm{EXP}$ (which has Δ_0-induction with bounded quantifiers) and considerably below PRA (full quantifier-free induction over arbitrary boolean combinations).

Caveat: step rule still uses propositional logic. While the induction *predicate* is atomic in all three sites, the induction *step* hypothesis fed to *ruleIndBT* has shape

$$P[O/v_1]\ \supset\ (P[O/v_2]\ \supset\ P[Pair(v_1, v_2)/0]),$$

an implication of three substitution-instances of the atomic predicate. Building and discharging it requires *mp* and the propositional axioms (*axK*, *axS*). Furthermore, the negation closure around *Con* (which has shape *not* (*atomic* ...)) requires classical propositional axioms: *axContrapos*, *axExFalso*, *axNeg*, used in `Thm11Diagonal.agda`, `Thm14Abstract.agda`, `Thm11.agda`, and `Thm14Numerals.agda`. These are not induction principles; they are classical propositional logic.

Combined calibrated headline. The two complementary audits give the following sharpened statement of the BRA2 result:

> **Theorem (calibrated).** Gödel's second incompleteness theorem is provable in BRA using only:

- *Functions:* the ramified-elementary fragment, $\Omega^k\theta \to \theta$ with $k \leq 2$ in Leivant's higher-type ramification — equivalent to Kalmar's elementary class $\mathcal{E}_3$;
- *Induction:* tree induction at atomic-equation predicates only — the BRA analog of open / quantifier-free induction;
- *Logic:* classical propositional logic (modus ponens, *axK*, *axS*, *axContrapos*, *axExFalso*, *axNeg*), equational congruence, and Hilbert-style instantiation.

No use of higher-type primitive recursion beyond level 2; no induction over compound formulas; no quantifiers.

The two fragment certificates are independent: the first restricts the function vocabulary (depth of *treeRec* nesting), the second restricts the predicate vocabulary (formula complexity at the induction site). Both are post-hoc certificates, observable by a finite `grep` audit of the seven function-defining files plus the three induction call sites; no Agda re-proof is required.

7.3 The threshold formal system: `DerivT`

The two calibrations of Sections **??** and **??** are observations: by inspection, the BRA2 proof of Gödel II uses only the ramified-elementary function fragment and only atomic-predicate tree induction. This subsection turns the second observation into a typed Agda artifact: a *threshold derivation type* `DerivT` (file `BRA2/DerivT.agda`) in which compound-formula tree induction is syntactically inadmissible. The full BRA derivation type `Deriv` remains unchanged; `DerivT` is a separate inductive family.

Definition of the threshold system. `DerivT : Formula -> Set` is defined identically to `Deriv` at every constructor except the binary-tree induction rules. Where `Deriv` has

```
ruleIndBT  : (P : Formula) (v1 v2 : Nat) ->
             Deriv (substF zero O P) ->
             Deriv ((substF zero (var v1) P) imp
                    ((substF zero (var v2) P) imp
                     (substF zero (ap2 Pair (var v1) (var v2)) P))) ->
             Deriv P
```

`DerivT` has the atomic-only restriction

```
indBT      : (e : Equation) (v1 v2 : Nat) ->
             DerivT (atomic (substEq zero O e)) ->
             DerivT ((atomic (substEq zero (var v1) e)) imp
                     ((atomic (substEq zero (var v2) e)) imp
                      (atomic (substEq zero (ap2 Pair (var v1) (var v2)) e)))) ->
             DerivT (atomic e)
```

and analogously for `ruleIndBT2` / `indBT2`. The predicate is now an `Equation`, so the user cannot supply a `not_` or `imp_` compound; the system is closed under atomic-predicate induction only. Every other constructor of `Deriv` (computation axioms, equational congruence, classical propositional axioms, modus ponens, `ruleInst`) is mirrored verbatim in `DerivT`.

Soundness: the forgetful embedding. `forget : DerivT P -> Deriv P` is defined by structural recursion: each `DerivT` constructor maps to the identically-named `Deriv` constructor, except `indBT e v1 v2 base step` translates to `ruleIndBT (atomic e) v1 v2 (forget base) (forget step)`, and analogously for `indBT2`. This is 40 cases of structural map. Soundness of the threshold system relative to BRA is thus a machine-checked theorem: anything provable in the threshold system is provable in BRA.

Concrete certificate at the three induction sites. The audit observation that all `ruleIndBT` / `ruleIndBT2` calls in the GoedelIIFull closure use atomic predicates is now a *type-level* fact, not merely a grep. Two derived rules

```
ruleIndBTAtomic  : (e : Equation) (v1 v2 : Nat) -> ... -> Deriv (atomic e)
ruleIndBT2Atomic : (e : Equation) (v1 v2 v3 v4 : Nat) -> ... -> Deriv (atomic e)
```

are added to `BRA2/Deriv.agda` (lines 273–316). These are type-level signatures for atomic-only `Deriv` induction, and their bodies just delegate to the underlying `ruleIndBT` / `ruleIndBT2` with motive `atomic e`. The three call sites in the closure

- `TreeEqReflParam.agda:158`,
- `Th12TreeEqUniv.agda:72`,
- `Th12TreeRecInternal.agda:1786`

have been refactored to use the `*Atomic` derived rules, taking an `Equation` parameter rather than an `atomic`-wrapped `Formula`. After the refactor a `grep` over the 147 files in the closure shows zero non-`*Atomic` invocations: the audit is enforced at the type level, not just by inspection.

Demonstrations. `BRA2/DerivTDemo.agda` typechecks four worked examples in the threshold system:

1. A bare leaf-leaf TreeEq fact (`axTreeEqLL`).
2. A small chained equational derivation (`ap1 I (ap1 I t) = t` via `cong1` and `ruleTrans`).
3. A direct use of `indBT` to derive `forall x. ap1 Z x = O` by atomic tree induction.
4. A direct use of `indBT2` to derive `forall x v. v = v` by 2D atomic tree induction with diagonal IHs.

Each example concludes with a `forget` call producing the corresponding `Deriv`, so both the constructive and the soundness directions are exercised.

The full lift, completed. The threshold-system version

$$godelII\_T : DerivT\ Con \to DerivT\ \bot$$

has been formalised. The closure of `GoedelIIFull.agda` (147 files) was migrated to a re-export module `BRA2/DerivThreshold.agda` that imports `BRA2.DerivT` with `DerivT` renamed to `Deriv` and `indBT`/`indBT2` renamed to `ruleIndBTAtomic`/`ruleIndBT2Atomic`, and re-ports all of `Deriv.agda`'s derived lemmas (axKT, axRecLf, axRecNd, DNE, axContrapos, axExFalso, etc.) onto the threshold type. Every `open import BRA2.Deriv` in the closure was replaced by `open import BRA2.DerivThreshold`;

constructor names match, so no proof body required restructuring. The eight remaining bare `ruleIndBT` invocations in `Thm12.Parts.{Fst,Snd,IfLf,TreeEq}` (all on atomic predicates, but using the compound-rule name) were refactored to `ruleIndBTAtomic` with explicit `Equation` witnesses. The `encodeRich` pattern matches in `Thm.ThmT.agda` were updated to the `*Atomic` constructors.

`GoedelIIFull.agda` now provides both signatures:

```
godelII   : Deriv  Con -> Deriv  bot   -- via DerivThreshold's renaming;
                                        -- semantically equivalent to godelII_T.
godelII_T : DerivT Con -> DerivT bot   -- explicit threshold-typed alias.
```

Both typecheck under `--safe --without-K --exact-split`. The original BRA derivation type `Deriv` in `BRA2/Deriv.agda` is left unchanged for use by out-of-closure files (`BRA2.DecodeAt`, the legacy `BRA2.Thm12.*` dispatchers that pre-date the threshold migration); the two systems coexist.

The conjunction of this lift and the function-class audit (Section **??**) is the formal threshold-system theorem:

> **Theorem (machine-checked threshold).** The threshold derivation type `DerivT` — BRA's quantifier-free equational calculus restricted to atomic-equation tree induction — proves
>
> $$\mathit{DerivT}\ \mathit{Con} \to \mathit{DerivT}\ \bot.$$
>
> The proof uses only the ramified-elementary fragment of the `Fun1`/`Fun2` primitives ($\Omega^k\theta \to \theta$ with $k \le 2$ in Leivant's higher-type ramification).

7.4 Towards bounded consistency reflection

The threshold system *DerivT* proves $\mathit{DerivT}\ \mathit{Con} \to \mathit{DerivT}\ \bot$ (Section **??**). By Gödel II this rules out *full* internal soundness: any putative $\mathit{decode\_at} : (P : \mathit{Formula}) \to \mathit{DerivT}\ (\mathit{atomic}\ (\mathit{eqn}\ (\mathit{ap}_1\ \mathit{thmT}\ \hat{P})\ (\mathit{codeFo}$ $\mathit{DerivT}\ P$ would, composed with $\mathit{godelII\_T}$, yield internal inconsistency. This blocks the strongest reflection statement.

What is not blocked is *bounded* reflection: a *DerivT* proof of $\bot$ whose proof-tree complexity is bounded by some syntactic measure can in principle be normalised to a proof of $\bot$ in an induction-free sub-language. Iterating the reduction lands in the open fragment, where consistency is observable. This is Nelson's strategy [**?**, **?**], deliberately stripped of his PA-inconsistency conjecture and stated for quantifier-free BRA.

We have laid down the typed scaffolding for this programme in three files (the Step 1–3-stub deliverables of `BRA2/BOUNDED-REFLECTION-PLAN.md`), all under `--safe --without-K --exact-split` and postulate-free:

- `BRA2/DerivT0.agda`: the open fragment DerivT_0, defined by mirroring every constructor of *DerivT except indBT* and indBT_2. A forgetful embedding $\mathit{forget}_0 : \mathit{DerivT}_0\ P \to \mathit{DerivT}\ P$ witnesses that open-fragment proofs lift into the threshold system. Open consistency is then $\mathit{OpenCon}_0 := \mathit{DerivT}_0\ \bot \to \emptyset$.

- `BRA2/DerivTBounded.agda`: an indexed family $\mathit{DerivTBounded}\ r\ l\ P$ tracking induction rank r and modus-ponens chain depth l. Computation axioms have rank 0, level 0; *mp* takes max on r and increments l; $\mathit{indBT}/\mathit{indBT}_2$ take max on l and increment r. A forgetful embedding $\mathit{forget}_B : \mathit{DerivTBounded}\ r\ l\ P \to \mathit{DerivT}\ P$ drops the indices. Bounded consistency is $\mathit{ConBounded}\ r\ l := \mathit{DerivTBounded}\ r\ l\ \bot \to \emptyset$.

- `BRA2/BoundedReduction.agda`: the proof obligation *as a type*,

$$\mathit{BoundedReductionTheorem} \;:=\; (r\, l : \mathbb{N}) \to \mathit{DerivTBounded}\; r\; l\; \bot \to \mathit{DerivT}_0\; \bot,$$

  together with the headline corollary $\mathit{BoundedReductionTheorem} \to \mathit{OpenCon}_0 \to \forall r\, l\; \mathit{ConBounded}\; r\; l$, proved in one line by contraposition. The file further *decomposes* the theorem by rank-induction: it proves $\mathit{RankDecrement} \to \mathit{BoundedReductionTheorem}$, where

$$\mathit{RankDecrement} \;:=\; \forall r\; l,\; \mathit{DerivTBounded}\; (\mathit{suc}\; r)\; l\; \bot \to \Sigma\, l',\; \mathit{DerivTBounded}\; r\; l'\; \bot$$

  is the *single-step* indBT-elimination at $\bot$. The body of *RankDecrement* is the actual technical heart of the programme and is left open.

- `BRA2/BoundedReductionRankZero.agda`: discharges the base case $\mathit{rankZero} : \forall l\, P,\; \mathit{DerivTBounded}\; 0\; l\; P \to \mathit{DerivT}_0\; P$ by structural recursion on the bounded derivation. All atomic axioms map directly; binary combiners (*mp*, *ruleTrans*) use a *natMaxZero* decomposition lemma to extract $r_1 = r_2 = 0$ from $\max(r_1, r_2) = 0$; $\mathit{indBT}/\mathit{indBT}_2$ are dismissed by absurdity of $\mathit{Eq}\; (\mathit{suc}\, k)\; 0$. This answers Q5 of plan Section 5.

Progress on *RankDecrement* at rank one. A substantial portion of the rank-1 case of *RankDecrement* is now in place. Given a $\mathit{DerivTBounded}\; 1\; l\; \bot$, we construct a $\mathit{DerivT}_0\; \bot$ for a clearly characterised slice of inputs. The construction is decomposed into three stages, each formalised in a separate Agda module under `--safe`, with no postulates and sub-second typecheck times.

1. **Locate.** A structural recursion descends through every rule of the rank-1 proof tree, locating the topmost induction step and accumulating the chain of intermediate equational reasoning between that induction step and the root as a *one-hole proof context*: a sequence of structural-rule frames (symmetry, transitivity, congruence, modus ponens, instantiation) recording how the induction's conclusion is transformed into $\bot$ along the path.

2. **Eliminate.** Two strategies handle the located induction step.

   When the equation under induction does not actually contain the bound variable free, the base premise is, by computation, already a derivation of the conclusion (since substituting the canonical value into a closed equation is the identity). Plugging this base premise through the recorded one-hole context yields a DerivT_0 derivation of $\bot$ directly, with no use of the induction-step premise.

   Otherwise — when the equation is genuinely open in the bound variable — we search the recorded context for an explicit *closing instantiation*: a frame substituting a value-shape term for the bound variable. When such a closing instantiation is reachable through any chain of single-step structural-rule wrappers (each wrapper commutes substitution past itself, with the fixed non-hole premises of binary frames substituted in turn), the standard induction-elimination procedure (a finite recursion guided by the closing value's tree shape) produces a DerivT_0 derivation at the substituted equation, after which the residual outer context plugs through to $\bot$.

3. **Dispatch and embed.** The two strategies are composed in a single pipeline: the closed-equation route is tried first because it bypasses the induction-step machinery; the open-equation route handles the residue. Composition with a structural embedding of DerivT_0 back into rank-0 *DerivTBounded* yields a function of the targeted *RankDecrement* shape,

modulo a Maybe wrapper recording the cases the locate- and-eliminate slice does not yet cover:

$$\mathit{DerivTBounded}\ 1\ l\ \bot\ \longrightarrow\ \mathsf{Maybe}\big(\Sigma\, l',\ \mathit{DerivTBounded}\ 0\ l'\ \bot\big).$$

Followed by *rankZero* and the open-consistency assumption $\mathit{OpenCon}_0$, this reduces a rank-1 proof of $\bot$ to a contradiction whenever the pipeline accepts the input.

The well-formedness side conditions of *indBT* (freshness of the bound variables, distinctness) are now *type-level invariants*: the bounded induction constructor itself takes a well-formedness record as an argument, so an ill-formed instance is unconstructible. This closes the corresponding side of the analysis at the type level rather than at runtime, removing one of the three previously identified sources of partial behaviour.

The remaining Maybe is genuinely partial: the pipeline returns nothing exactly when the closing instantiation exists in the proof tree but is buried beneath a nested combination of wrappers that the current single-step rewriting does not unfold (for instance, a doubled symmetry frame or an instantiation inside an instantiation), or when the original tree uses the two-variable tree-pair induction primitive indBT_2 rather than the single-variable *indBT*. Both of these are recognised cases of the same finite case analysis already handled for the single-step wrappers, and are scheduled as the next concrete extensions.

The general higher-rank case ($r \to r-1$ for $r > 1$) remains conjectural; its difficulty is precisely whether the same elimination procedure stays inside the rank-r bound or escapes it, and is the central calibration question for the complexity measure.

A panel of thirteen refl-witnessed end-to-end tests runs the full pipeline on synthetic rank-1 inputs whose induction premises are taken as abstract parameters (we cannot construct them concretely without contradicting consistency). Each test reduces to just definitionally, locking in that the certified pipeline does not merely typecheck but actually computes through to a DerivT_0 derivation of $\bot$.

The selection of complexity measure (induction rank, checker rank, reflection rank, or a Grzegorczyk-class rank tied to the function hierarchy of Section **??**) is itself the research question and must be made jointly with the proof attempt: the metric is the one the reduction procedure decreases. The default starting point is the simple induction-rank bookkeeping shipped in `DerivTBounded.agda`.

If the bounded reduction theorem is proved with a measure tied to ε_3 resources (Section **??**) it remains inside the threshold fragment and the conclusion is itself a *DerivT* theorem. If the reduction requires super-elementary resources, the bound moves out of the threshold and the resulting consistency reflection becomes a meta-theorem only. This is the falsification criterion for the present complexity calibration.

7.5 Comparison with prior formalisations

The only previous machine formalisation of Gödel's *Second* Incompleteness Theorem we are aware of is Paulson's Isabelle/HOL development of hereditarily finite set theory [**?**], following Świerczkowski [**?**]. Other formalisations (O'Connor in Coq, Han in Lean, Carneiro's metamath work) target Gödel's *First* theorem only.

Paulson's development uses HF set theory and full Gödel numbering of Hilbert-style proofs, with encoded substitution treated as an external soundness theorem. Bound variables in the formal language (HF has $\exists$ and $\forall$ as primitive binders, with bound variables significant for inductive proofs over formula structure) are handled via *Nominal Isabelle* [**?**]: a non-standard extension of Isabelle/HOL that quotients binding constructions modulo α-equivalence, with permutations on names as the underlying mechanism. Paulson reports that the nominal framework is essential to

make induction over formula structure tractable (O'Connor's first-incompleteness Coq formalisation needed roughly 1900 lines for a single substitution lemma without nominal support); the cost is that any function defined on formulas must be proved to use bound variables "sensibly," a property whose proofs are demanding and required dedicated assistance from Christian Urban. For coding, Paulson separately uses de Bruijn indices and proves a correspondence with the nominal syntax.

The present development sidesteps this entire issue. BRA's *Formula* type has no quantifiers — only atomic equations, $\neg$, and $\supset$ — so there are no bound variables in the formal language at all; what appears in *atomic* (*eqn* ℓ r) are just object-level Terms, with free variables denoted *var* n for $n : \mathbb{N}$. Universal closure of an open formula is expressed implicitly via the BRA rule *ruleInst* (substitution into a derived formula), without any meta-level renaming. Substitution at the term level is the ordinary primitive-recursive *subst* : $\mathbb{N} \to \mathit{Term} \to \mathit{Term} \to \mathit{Term}$, capture-free by construction (no binder under which to capture). Coded substitution *sub* : Fun_2 is itself a BRA functor; its correctness is the equational *codeCommutesFormula* lemma, not a metatheoretic substitution-lemma proof. In short, what required Nominal Isabelle in Paulson's HF formalisation is for us a non-issue, because Guard's BRA was designed without binding constructors in the first place.

Paulson's "mistake" in Boolos–Świerczkowski, and Guard's analogous construction. Paulson devotes §6 of his paper to identifying a presentational flaw in Boolos [**?**] and Świerczkowski's expositions of the second incompleteness theorem in HF. Both sources describe the internal substitution as $\mathrm{Pf}\lfloor\alpha\rfloor_V(Q)$, read informally as "$\alpha$ with each free variable x_i replaced by the formal term $Q(x_i)$," where Q is a "pseudo-function" mapping a value to its name in the formal language. Paulson's critique: Q is not a function of the formal language (HF has only one function symbol, $\lhd$), and any version of the substitution that leaves the original variables x free in the transformed formula does not work — because x ranges over all HF sets, including codes of formulas, and the supposed equation $Q(x)$ = "the numeral of x" has no formal status. Paulson's corrected version introduces *new* free variables x' that appear in the transformed formula, constrained by relational antecedents $\mathrm{QR}(x, x')$ on the left of the turnstile. In §7 he further shows that Świerczkowski's earlier attempt to define such a Q via a total ordering on the HF universe has a significant gap, which Paulson closes by working with the relation QR throughout instead of the function Q.

The same conceptual issue arises in Guard's notation in `guard15.pdf`. Guard writes the encoded equational defining clause for a primitive functor f as

$$\mathit{th}(D_f\, x) \;=\; {}^{\ulcorner} f(\underline{x}) \;=\; \underline{f(x)}^{\urcorner},$$

where the underline $\underline{(\cdot)}$ takes a value to its numeral term — exactly Boolos–Świerczkowski's pseudo-function Q at a different name. At a free variable x this notation is not a BRA expression: there is no operator in BRA's term language that, given an arbitrary Term, returns "the numeral of its value" (Section **??**). Read literally, Guard's $\underline{x}$ would be the same kind of pseudo-function Paulson criticises.

The BRA chain treats this rigorously the way Paulson treats his QR. The combinator *cor* : Fun_1 is a genuine BRA functor (Layer 3, BRA2/Cor.agda) with a primitive recursive defining equation; for value-shaped inputs x it satisfies $\mathit{ap}_1\ \mathit{cor}\ x \;=\; \mathit{code}\ x$ internally (the lemma *corOfReify*). At free variables it does *not* reduce, just as Paulson's $\mathrm{QR}(x, x')$ does not equate x with anything specific: instead it constrains x' relationally. Theorem 14 step 3 in our development then proves internal-implication statements

$$\mathit{thmT}\, x = \mathit{codeFormula}\, G \;\;\to\;\; \mathit{thmT}\,(g\_\mathit{part}\, x) = \mathit{th}\,\widehat{\hat{x} = \mathit{sub}\, j\, j},$$

parametric in x, with the $\hat{x} = cor\, x$ slot left syntactic and the closure transporting it to $code\, x$ only at the value-shaped instantiation step (the encode-witness, certified by *IsValue*). This is structurally the same move as Paulson's: introduce parametric new variables, constrain by relation on the antecedent side, never assert "$\hat{x}$ equals the numeral of x" as an unconditional formal claim.

A further connection to Paulson's §7: Świerczkowski needed an ordering on the HF universe to define Q by recursion on rank, and the resulting construction had a gap. In Guard's BRA, *cor* requires no such ordering — it is built directly as *Rec stepCor* from BRA's primitive-recursion combinator on binary trees (BRA2/Cor.agda), with *stepCor* a compact *Fan+Lift+KT+Post+Pair* expression. The primitive-recursive structure of trees is enough; no recursion on rank, no ordering, no gap.

The present development follows Guard [**?**]: the "provability predicate" $thmT$ is itself a primitive-recursive combinator built from $Rec + Fan + Lift + IfLf$; encoded substitution is performed by the explicit primitive recursive functor $sub : Fun_2$; the entire chain stays first-order; and the proof is equational at the encoded layer rather than provability-logical at the formula layer (Section **??**).

References

[1] J. R. Guard, *Lecture notes on recursive arithmetic*, Applied Mathematics Division, Argonne National Laboratory, 1963.

[2] L. C. Paulson, *A machine-assisted proof of Gödel's incompleteness theorems for the theory of hereditarily finite sets*, Review of Symbolic Logic, vol. 7 no. 3, 2014, pp. 484–498.

[3] S. Świerczkowski, *Finite sets and Gödel's incompleteness theorems*, Dissertationes Mathematicae, vol. 422, Polish Academy of Sciences, 2003.

[4] G. S. Boolos, *The Logic of Provability*, Cambridge University Press, 1993.

[5] C. Urban and C. Kaliszyk, *General bindings and alpha-equivalence in Nominal Isabelle*, Logical Methods in Computer Science, vol. 8 no. 2, 2012.

[6] R. Davies and F. Pfenning, *A modal analysis of staged computation*, Journal of the ACM, vol. 48 no. 3, May 2001, pp. 555–604.

[7] D. Leivant, *Ramified recurrence and computational complexity III: Higher type recurrence and elementary complexity*, Annals of Pure and Applied Logic, vol. 96 no. 1–3, 1999, pp. 209–229.

[8] E. Nelson, *Predicative Arithmetic*, Mathematical Notes 32, Princeton University Press, 1986.

[9] E. Nelson, *Elements* (unfinished draft on inconsistency of Peano Arithmetic, withdrawn after Tao–Tausk gap), 2011. Cited only for methodology; the substantive PA-inconsistency claim is not endorsed.

[10] A. J. Wilkie and J. B. Paris, *On the scheme of induction for bounded arithmetic formulas*, Annals of Pure and Applied Logic, vol. 35, 1987, pp. 261–302. See also S. R. Buss, *Bounded Arithmetic*, Bibliopolis, 1986.